

SOFTWARE DESIGN DOCUMENT

Online Voting System

Web-Based Application

Submitted by

Muhammad Abdullah Aqeel

Roll No: S25BINFT7M04018

Submitted to

Dr. Suleman Shah

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Session: 2024–2025

1. Introduction

This Software Design Document (SDD) translates the Online Voting System's SRS requirements into a concrete architectural and component-level design blueprint. It covers system architecture, component breakdown, database schema, API design, security design, and deployment architecture.

1.1 Purpose

To provide the development team with a precise technical specification that enables consistent, standards-compliant implementation of the Online Voting System, covering all five functional requirements.

1.2 Scope

Full three-tier architecture: React.js frontend, Node.js/Express REST API backend, MongoDB + Redis data tier. All security, anonymity, and audit requirements from the SRS are addressed in this document.

1.3 Conventions

Term	Meaning
SDD	Software Design Document
JWT	JSON Web Token — stateless session token
OTP	One-Time Password (6-digit, 5-minute validity)
MFA	Multi-Factor Authentication (password + OTP)
TOTP	Time-based One-Time Password (RFC 6238)
CNIC	Computerised National Identity Card (Pakistan)
E2E	End-to-End (encryption / verifiability)
FR	Functional Requirement
NFR	Non-Functional Requirement

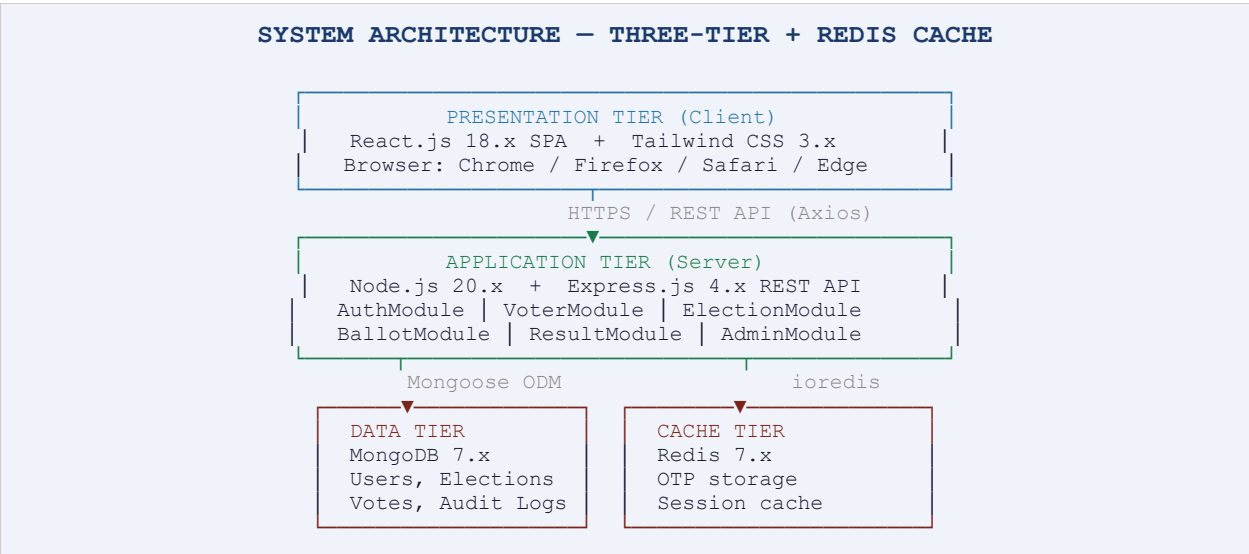
1.4 Intended Audience

Developer (Muhammad Abdullah Aqeel), Supervisor (Dr. Suleman Shah), External Examiner, and future maintainers of the codebase.

2. System Architecture

The system uses a Three-Tier Architecture with an additional Redis Cache Tier for OTP storage and session caching. Each tier is independently deployable and communicates via clearly defined interfaces.

Figure 1 — System Architecture Diagram:



2.1 Architecture Layers

Tier	Technology	Responsibility
Presentation	React.js + Tailwind	UI, ballot display, result charts, voter dashboard
Application	Node.js + Express.js	Business logic, REST API, auth, vote processing
Data	MongoDB 7.x	Persistent storage of all documents/collections
Cache	Redis 7.x	OTP storage (TTL 5 min), session caching

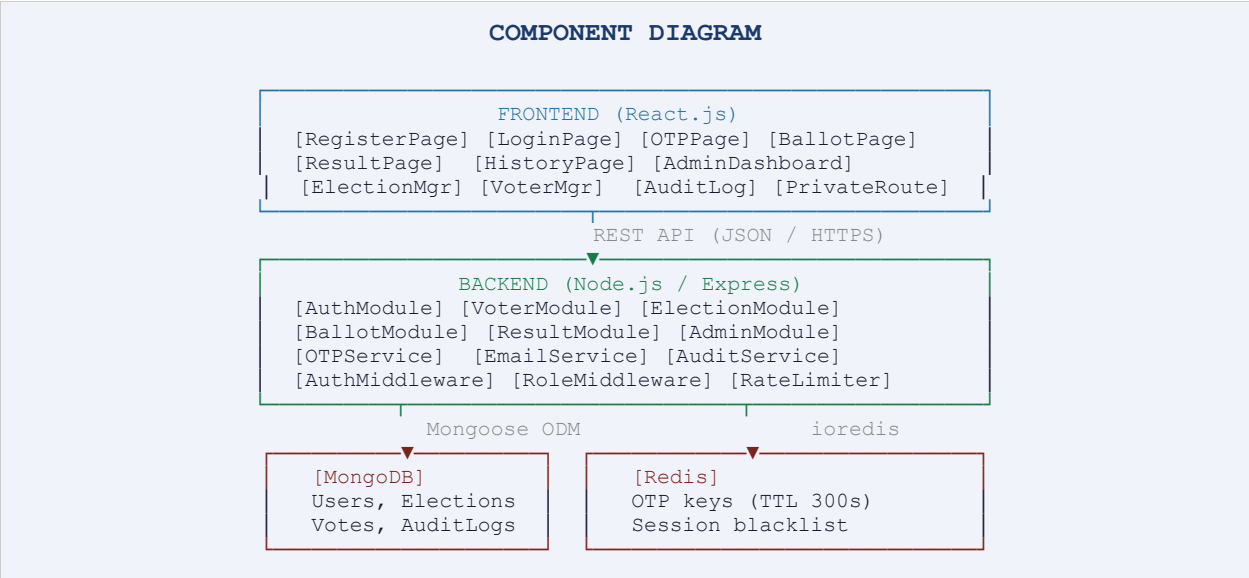
2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	SPA component-based UI
Frontend	Tailwind CSS	3.x	Responsive utility-first styling
Backend	Node.js / Express.js	20.x / 4.x	REST API server & routing
Backend	JWT + Bcrypt	Latest	Auth tokens + password hashing
Backend	Nodemailer	Latest	OTP delivery via email
Database	MongoDB	7.x	NoSQL document store

Cache	Redis	7.x	OTP store (TTL), session cache
Security	Helmet.js / CORS	Latest	HTTP headers & cross-origin policy
DevOps	Docker + GitHub Actions	Latest	Containerisation + CI/CD pipeline

3. Component Design

Figure 2 — Component Diagram:



3.1 Frontend Components

Component	Responsibility
RegisterPage	Voter registration form with CNIC, email, password; real-time validation
LoginPage	Email + password login; triggers OTP dispatch on success
OTPPage	6-digit OTP input; 5-min countdown; resend option
BallotPage	Displays active election candidates; enforces single selection; confirmation prompt
ResultPage	Bar chart of vote counts, winner badge, turnout percentage; public/private toggle
AdminDashboard	Stats overview: total voters, active elections, votes cast today
ElectionMgr	Create/edit elections; add candidates; state transition controls
VoterMgr	Approve/suspend voters; import CSV voter roll
AuditLog	Filterable tamper-evident log of all system events
PrivateRoute	HOC: redirects unauthenticated or unauthorised users

3.2 Backend Modules

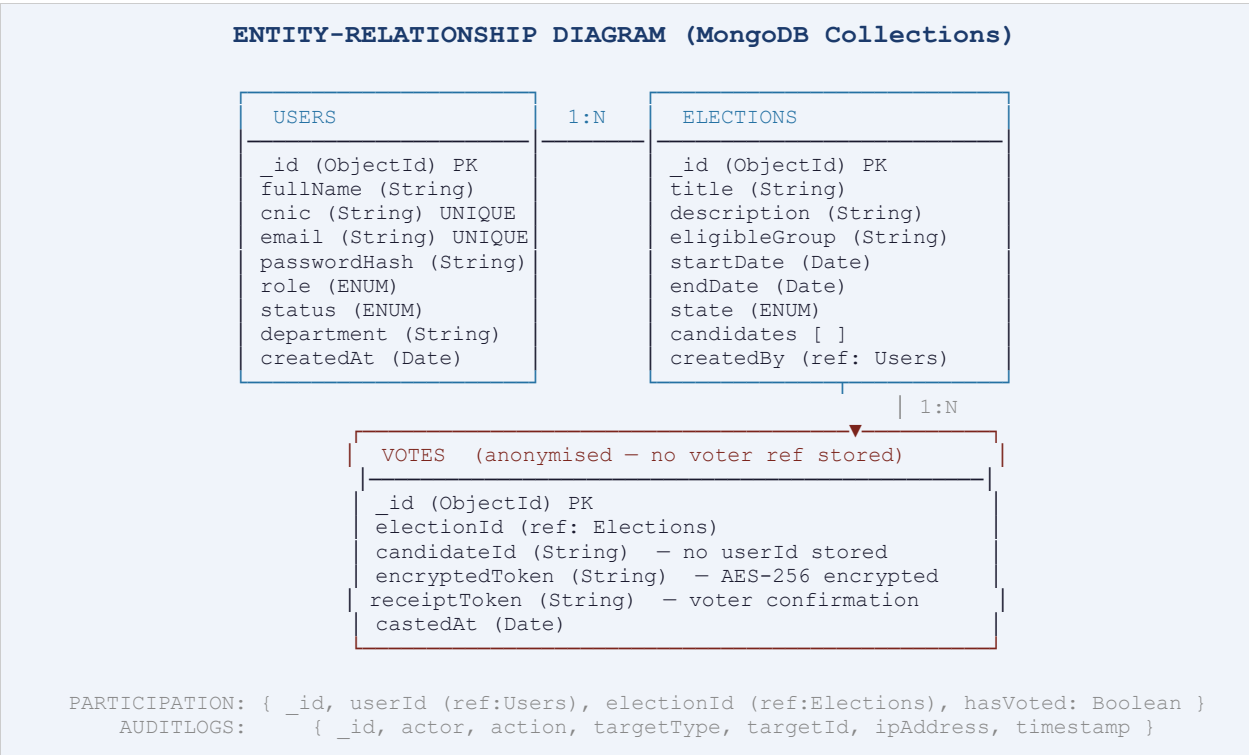
Module / Service	Responsibility
AuthModule	POST /register, POST /login, POST /verify-otp — bcrypt, OTP, JWT issuance
VoterModule	Voter profile, participation history, status check

ElectionModule	CRUD for elections; state machine: Draft → Active → Closed
BallotModule	Eligibility check, duplicate vote check, vote encryption + anonymisation, receipt generation
ResultModule	Tally aggregation, percentage calc, winner determination, result publication
AdminModule	Voter approval/suspension, CSV import, audit log queries, report download
OTPService	Generates TOTP; stores in Redis with 300s TTL; validates on submission
EmailService	Sends OTP email via Nodemailer; sends approval/rejection notifications
AuditService	Writes tamper-evident log entries for all critical events
AuthMiddleware	Validates JWT on all protected routes; checks token blacklist in Redis
RateLimiter	100 req/15 min per IP on auth routes; blocks brute-force attacks

4. Database Design

The system uses MongoDB 7.x with a document-oriented schema. Vote anonymity is enforced by design: the VOTES collection stores no reference to the voter's identity — only the election and candidate.

Figure 3 — Entity-Relationship Diagram:



4.1 Collection Definitions

4.1.1 USERS Collection

Field	Type / Constraint	Notes
<code>_id</code>	ObjectId	MongoDB auto-generated primary key
<code>fullName</code>	String, required	Minimum 3 characters
<code>cnic</code>	String, unique	13-digit format; unique index enforced
<code>email</code>	String, unique	Institutional email; login identifier
<code>passwordHash</code>	String	bcrypt hash, cost factor 12
<code>role</code>	Enum: USER ADMIN	Default: USER
<code>status</code>	Enum: PENDING ACTIVE SUSPENDED	Default: PENDING_APPROVAL
<code>department</code>	String	Voter's constituency or department group
<code>createdAt</code>	Date	Auto-set on document creation

4.1.2 ELECTIONS Collection

Field	Type / Constraint	Notes
_id	ObjectId	Auto-generated PK
title	String, required	Election name displayed to voters
state	Enum: DRAFT ACTIVE CLOSED	State machine; only valid transitions allowed
eligibleGroup	String	Links to voter department/constituency
startDate / endDate	Date	Controls voting window; auto-transition hook
candidates	Array of subdocs	Each: { candidateId, name, party, photoUrl }
createdBy	ObjectId ref: Users	Admin who created the election

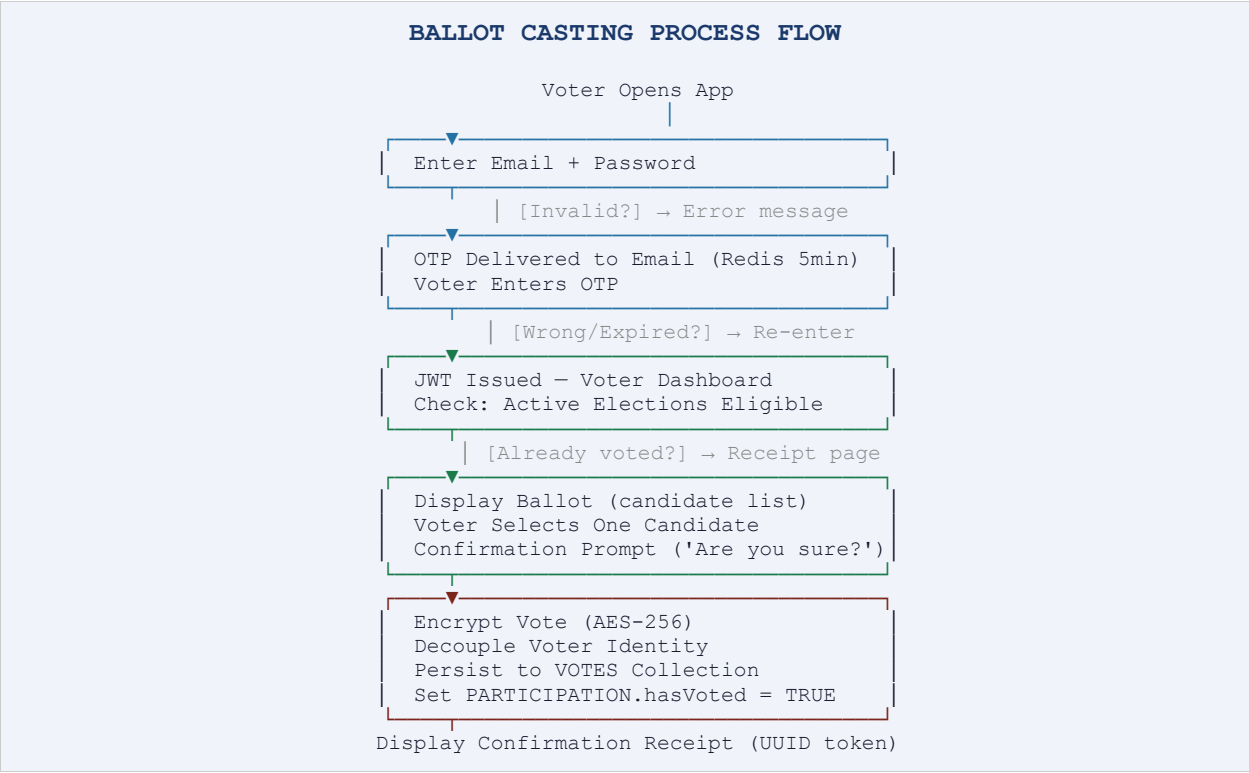
4.1.3 VOTES Collection (Anonymised)

Field	Type / Constraint	Notes
_id	ObjectId	Auto-generated PK
electionId	ObjectId ref: Elections	Which election this vote belongs to
candidateId	String	ID of the chosen candidate
encryptedToken	String (AES-256)	Encrypted vote token — NO voter identity stored
receiptToken	String (UUID)	Issued to voter as confirmation proof
castedAt	Date	Submission timestamp

Key Design Decision: No userId is stored in the VOTES collection. Anonymity is enforced by the PARTICIPATION collection which only records that a voter has voted (boolean flag) — never which candidate they chose.

5. API Design & Process Flow

Figure 4 — Ballot Casting Process Flow Diagram:



5.1 REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new voter account (PENDING status)
POST	/api/auth/login	Public	Validate credentials; trigger OTP dispatch to email
POST	/api/auth/verify-otp	Public	Verify OTP; issue JWT access token on success
GET	/api/elections/active	JWT	List active elections eligible for the voter
GET	/api/elections/:id/ballot	JWT	Fetch ballot (candidates) for a specific election
POST	/api/elections/:id/vote	JWT	Submit encrypted anonymised vote; returns receipt token
GET	/api/elections/:id/results	JWT / Public	Fetch results for a closed election
GET	/api/admin/voters	ADMIN JWT	List all voters with status; filter by department
PATCH	/api/admin/voters/:id	ADMIN JWT	Approve / suspend / delete voter account
POST	/api/admin/elections	ADMIN JWT	Create new election in DRAFT state
PATCH	/api/admin/elections/:id/state	ADMIN JWT	Transition election: DRAFT→ACTIVE→CLOSED
GET	/api/admin/audit-logs	ADMIN JWT	Paginated tamper-evident audit log; filterable

POST	/api/admin/voters/import-csv	ADMIN JWT	Bulk import voter roll from CSV file
------	------------------------------	-----------	--------------------------------------

6. Security Design

Vote integrity and voter anonymity are the two most critical non-functional properties of this system. The design below addresses all OWASP Top 10 threats and the specific requirements of NFR004 through NFR008.

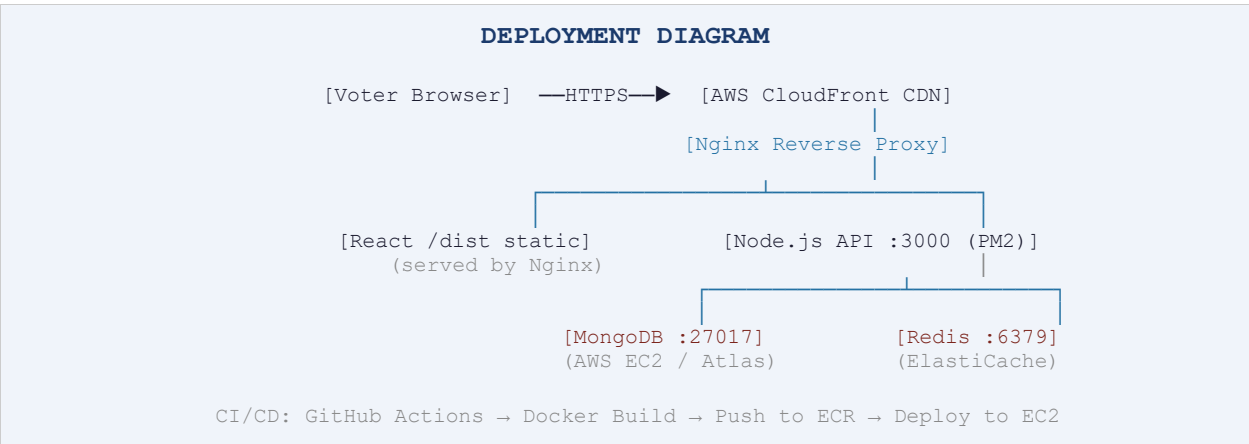
Security Layer	Mechanism	Detail
MFA Authentication	Password + Email OTP	bcrypt (cost 12) + TOTP stored in Redis, 300s TTL
Session Management	JWT HS256	24h expiry; logout blacklists token in Redis
Vote Anonymity	Identity Decoupling	VOTES collection stores no userId; only PARTICIPATION tracks who has voted
Vote Encryption	AES-256 CBC	Each vote encrypted before DB persistence; key stored in env variable
Transport Security	HTTPS / TLS 1.3	All client-server traffic encrypted end-to-end
Duplicate Vote	DB unique index + app check	Unique compound index on (userId, electionId) in PARTICIPATION
Brute Force	express-rate-limit	100 req/15 min on /auth/* routes; account lockout after 5 fails
Input Validation	express-validator + Joi	All inputs sanitised; NoSQL injection prevented
HTTP Security	Helmet.js	Sets CSP, X-Frame-Options, HSTS, XSS-Protection headers
CSRF Protection	SameSite=Strict + CORS	CORS restricted to frontend origin; state-changing via JSON body only
Audit Logging	AUDITLOGS collection	All auth events, votes, admin actions logged with IP and timestamp

7. Non-Functional Requirements — Design Mapping

ID	Category	Design Decision
NFR001	Performance	Redis caches session data; MongoDB indexed queries; vote response target < 3s
NFR002	Availability	PM2 cluster mode + Docker; AWS EC2 with auto-recovery; 99.9% uptime SLA
NFR003	Scalability	Stateless JWT enables horizontal scaling; MongoDB replica set; Redis for shared session state
NFR004	Security	HTTPS, MFA, JWT, AES-256 vote encryption, Helmet.js, rate limiting — OWASP Top 10 compliant
NFR005	Integrity	Unique compound index (userId, electionId) in PARTICIPATION; app-level + DB-level duplicate check
NFR006	Anonymity	VOTES collection stores zero voter identity; PARTICIPATION only records boolean hasVoted flag
NFR007	Usability	Tailwind responsive grid; WCAG 2.1 AA; tested Chrome, Firefox, Safari, Edge desktop + mobile
NFR008	Auditability	AUDITLOGS collection: every auth event, vote, admin action — IP, timestamp, actor, target

8. Deployment Architecture

Figure 5 — Deployment Diagram:



Component	Deployment Detail
Frontend	React production build served as static files via Nginx; cached by CloudFront CDN
Backend API	Node.js managed by PM2 in cluster mode; listens on port 3000; behind Nginx reverse proxy
MongoDB	MongoDB Atlas (cloud) or self-hosted on AWS EC2 with replica set for high availability

Redis	AWS ElastiCache or self-hosted Redis 7.x; used for OTP TTL storage and JWT blacklist
CI/CD	GitHub Actions: lint → test → Docker build → push to AWS ECR → deploy to EC2
SSL	Let's Encrypt certificate managed by Certbot; auto-renew; TLS 1.3 enforced at Nginx

